



SERENITA

Vero
Security Assessment Report

Version: 2.0

April, 2026

Contents

Introduction	2
Executive Summary	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	3
Findings Summary	3
Detailed Findings	4
Summary of Findings	5
Checkpoint Confirmation Cache Not Reset On Chain Reorganisation	6
Critical Operations Rely On Single Beacon Node	9
Stale RANDAO Reveal May Lead To Invalid Block	12
Incorrect MEV Registration Leads To Potential Income Loss	14
Missing Signing Timeout Can Block All Attestation Publication	16
Strict Slot Equality May Drop Valid Attestations	18
Non-Atomic Duty Refresh Can Cause Missed Attestations	20
Overly Permissive Permissions On Keymanager API Token File	22
Retired <code>RemoteSigner</code> Instances Are Never Closed	23
Single Beacon Node Can Trigger False Slashing Halt	26
Unbounded Provider Response Reads Can Exhaust Memory	28
Keymanager API Returns Non-Compliant Error Status Codes	30
Stale MEV Relay Registration Before Block Proposal	32
Head Event Dependent Root Checks Are Not Epoch-Specific	34
Pre-Emptive Duty Consumption Can Cause Irrecoverable Slot Loss On Proposal Failure	36
Request-Level <code>ClientTimeout</code> Usage Extends Effective Timeouts	38
Miscellaneous General Comments	41
A Vulnerability Severity Classification	43

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Serenita components in scope. The review focused solely on the security aspects of these components, though general recommendations and informational comments are also provided.

Executive Summary

This review identified 16 findings, with no High or Critical severity issues in scope. The most significant risks relate to single-node trust assumptions in safety-critical paths, chain-reorganisation handling gaps, and resilience weaknesses that can lead to missed attestations or proposals under adverse conditions. While these issues do not indicate direct key compromise, they can materially affect validator safety and liveness, including reward loss and extended duty disruption.

Priority remediation should focus on reducing reliance on single beacon node inputs for critical decisions, making cache and duty lifecycle logic explicitly reorg-aware, and enforcing strict timeout and resource bounds for remote dependencies such as beacon nodes and signers. Secondary hardening should address operator-facing security defaults and standards compliance gaps in the Keymanager API surface.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Serenita components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open / closed / resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Serenita components in scope.

Overview

Vero is a multi-node validator client for Ethereum and Gnosis Chain. It is designed to reduce single-client risk by comparing chain data across multiple Consensus Layer and Execution Layer client pairs before performing validator duties (for example, attestations). Vero is built to integrate with remote signers, rather than holding validator keys directly. Vero depends upon the anti-slashing guarantees provided by the remote signer, and does not implement internal slashing protection or key management features. Vero emphasises a small, auditable codebase with strong observability and broad client compatibility.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [serenita-org/vero](https://github.com/serenita-org/vero) repository.

The scope of this time-boxed review was strictly limited to files at commit [ca20c0e](#).

The fixes of the identified issues were assessed at commit [61d2e70](#).

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Python.

The manual review focused on identifying issues associated with the business logic implementation of the modules and services in scope. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Python runtime and its asynchronous execution model.

Additionally, the manual review process focused on identifying vulnerabilities related to known Python anti-patterns and attack vectors, such as race conditions in asynchronous code, non-atomic state mutations, cache invalidation failures, resource lifecycle management (leaked connections and background tasks), unbounded memory consumption from unvalidated external input, improper timeout and deadline handling, overly permissive file permissions, and incorrect trust assumptions when coordinating with external services.

To support the Python components of the review, the testing team may use the following automated testing tools:

- Bandit: <https://github.com/PyCQA/bandit>
- Pylint: <https://github.com/pylint-dev/pylint>
- mypy: <https://github.com/python/mypy>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 17 issues during this assessment. Categorised by their severity:

- Medium: 6 issues.
- Low: 7 issues.
- Informational: 4 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Serenita components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open**: the issue has not been addressed by the project team.
- **Resolved**: the issue was acknowledged by the project team and updates to the affected components(s) have been made to mitigate the related risk.
- **Closed**: the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
VERO-01	Checkpoint Confirmation Cache Not Reset On Chain Reorganisation	Medium	Resolved
VERO-02	Critical Operations Rely On Single Beacon Node	Medium	Resolved
VERO-03	Stale RANDAO Reveal May Lead To Invalid Block	Medium	Resolved
VERO-04	Incorrect MEV Registration Leads To Potential Income Loss	Medium	Resolved
VERO-05	Missing Signing Timeout Can Block All Attestation Publication	Medium	Resolved
VERO-06	Strict Slot Equality May Drop Valid Attestations	Medium	Resolved
VERO-07	Non-Atomic Duty Refresh Can Cause Missed Attestations	Low	Resolved
VERO-08	Overly Permissive Permissions On Keymanager API Token File	Low	Resolved
VERO-09	Retired RemoteSigner Instances Are Never Closed	Low	Resolved
VERO-10	Single Beacon Node Can Trigger False Slashing Halt	Low	Closed
VERO-11	Unbounded Provider Response Reads Can Exhaust Memory	Low	Resolved
VERO-12	Keymanager API Returns Non-Compliant Error Status Codes	Low	Resolved
VERO-13	Stale MEV Relay Registration Before Block Proposal	Low	Resolved
VERO-14	Head Event Dependent Root Checks Are Not Epoch-Specific	Informational	Closed
VERO-15	Pre-Emptive Duty Consumption Can Cause Irrecoverable Slot Loss On Proposal Failure	Informational	Closed
VERO-16	Request-Level ClientTimeout Usage Extends Effective Timeouts	Informational	Closed
VERO-17	Miscellaneous General Comments	Informational	Resolved

VERO-01 Checkpoint Confirmation Cache Not Reset On Chain Reorganisation			
Assets	src/providers/attestation_data_provider.py src/providers/beacon_node.py src/services/attestation.py src/initialize.py		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The checkpoint confirmation cache in `AttestationDataProvider` is never invalidated during onchain reorganisation events, allowing previously confirmed but now-stale finality checkpoints to bypass multi-node verification. When triggered, validators sign attestations with incorrect source or target checkpoints, causing them to vote for a non-canonical chain fork in Casper FFG.

`AttestationDataProvider` maintains two dictionaries each keyed by epoch: `source_checkpoint_confirmation_cache` and `target_checkpoint_confirmation_cache`. When `_confirm_finality_checkpoints()` is called, it checks whether the incoming source and target checkpoints match the cached values for their respective epochs. On a cache hit, it returns immediately without querying any other beacon node:

```
src/providers/attestation_data_provider.py::AttestationDataProvider._confirm_finality_checkpoints()

async def _confirm_finality_checkpoints(
    self,
    source: SchemaBeaconAPI.Checkpoint,
    target: SchemaBeaconAPI.Checkpoint,
    slot: int,
) -> None:
    if source == self.source_checkpoint_confirmation_cache.get(
        source.epoch
    ) and target == self.target_checkpoint_confirmation_cache.get(target.epoch):
        # @audit Returns without multi-node confirmation
        return

    # ... multi-node confirmation via wait_for_checkpoints() ...
    self._cache_checkpoints(source=source, target=target)
```

The cache is populated after successful multi-node confirmation of checkpoints in `_confirm_finality_checkpoints()` or after multi-node attestation data consensus in `_produce_attestation_data_without_expected_head_block_root()`, both of which require consensus exceeding the configured threshold across all beacon nodes. Once populated, entries persist until the periodic prune job runs, every 10 minutes, retaining 3 entries. `AttestationDataProvider` is not registered as a reorg event handler in `_register_event_handlers()` within `initialize.py`, so the cache is never cleared when a chain reorganisation occurs.

The attestation service also eagerly populates the cache. In `attest_if_not_yet_attested()`, even when no attestation duty exists for the current slot, attestation data is produced and checkpoints are cached if duties exist later in the epoch:

```
src/services/attestation.py::AttestationService.attest_if_not_yet_attested()
```

```
else:
    # Produce attestation data if there is an attester
    # duty scheduled for later in the epoch.
    # This ensures finality checkpoints are confirmed and cached early
    # into the epoch, even with a low number of active validators.
    epoch = slot // self.beacon_chain.SLOTS_PER_EPOCH
    if len(self.attester_duties[epoch]) > 0:
        _ = await asyncio.wait_for(
            self.attestation_data_provider.produce_attestation_data(
                # ... snip ...
            ),
            # ... snip ...
        )
```

This means checkpoints are typically cached from the first slot of an epoch, and all subsequent attestations in that epoch rely on the cache rather than live multi-node confirmation.

If a reorganisation occurs mid-epoch and a stale beacon node emits the first head event, Vero fetches attestation data exclusively from that stale node — `wait_for_attestation_data()` only accepts data matching the head event's block root:

```
src/providers/beacon_node.py::BeaconNode.wait_for_attestation_data()
```

```
async def wait_for_attestation_data(
    self,
    expected_head_block_root: str,
    slot: int,
) -> SchemaBeaconAPI.AttestationData:
    while True:
        # ... snip ...
        try:
            _, att_data = await self.produce_attestation_data(
                slot=slot,
            )
            # @audit Only the stale node can match a pre-reorg block root
            if att_data.beacon_block_root == expected_head_block_root:
                self.logger.debug(f"Got matching AttestationData from {self.host}")
                return att_data
        # ... snip ...
```

The stale checkpoints hit the cache, bypass multi-node confirmation, and the validator signs an attestation voting for the pre-reorg fork. This block root filter does provide implicit protection: the scenario specifically requires the stale node's head event to arrive first, since it cannot match a post-reorg block root.

This issue has a high impact because when the reorganisation changes the justified epoch (source checkpoint epoch), the stale attestation can trap the validator in a surround vote. If the stale chain had justified epoch N but the canonical chain reverts to $N-1$, the stale attestation is recorded as (source= N , target=current_epoch). Any future correct attestation on the canonical chain has source= $N-1$, which is less than N , and a later target epoch, satisfying the surround vote condition. With functioning slashing protection the validator is locked out of attesting until the canonical justified epoch advances past N , which could span many epochs during finality failures. Without functioning slashing protection, the surround vote is signed and the validator is slashed. In the less severe case where only the target root changes (same epochs), no surround vote is created but attestation rewards are still lost.

This issue has a low likelihood because it requires a chain reorganisation that alters checkpoint roots, combined with a stale beacon node winning the head event race. The surround vote scenario specifically requires a reorg that changes the justified epoch, which occurs during deeper reorgs or finality failures.

Recommendations

Register `AttestationDataProvider` as a reorg event handler in `_register_event_handlers()` and clear both `source_checkpoint_confirmation_cache` and `target_checkpoint_confirmation_cache` when a `ChainReorgEvent` is received. This preserves the performance benefit of caching during stable chain operation while restoring multi-node protection during reorganisations, which is precisely when it matters most.

Resolution

The development team has registered `AttestationDataProvider` as a reorg event handler in `_register_event_handlers()` within `initialize.py`, and added a `handle_reorg_event()` method that conditionally clears both checkpoint confirmation caches. The caches are only cleared when the reorg depth exceeds the epoch boundary, which is a more precise condition that avoids unnecessary cache invalidation while still addressing the root cause.

This issue has been resolved in commit [41f9527](#).

VERO-02 Critical Operations Rely On Single Beacon Node			
Assets	src/providers/multi_beacon_node.py src/providers/doppelganger_detector.py src/initialize.py src/services/attestation.py src/services/block_proposal.py		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

Several critical operations, including doppelganger detection and duty fetching, rely on a single beacon node instead of reconciling data across multiple nodes. This can degrade both safety and liveness.

The `MultiBeaconNode` class exposes a `best_beacon_node` property that selects a single beacon node based on the highest score:

```
src/providers/multi_beacon_node.py::MultiBeaconNode.best_beacon_node()

@property
def best_beacon_node(self) -> BeaconNode:
    return next(
        bn
        for bn in sorted(
            self.initialized_beacon_nodes, key=lambda bn: bn.score, reverse=True
        )
    )
```

This single node is used as the sole source for both duty fetching and doppelganger detection, creating two distinct failure modes.

The `DoppelgangerDetector` is initialised with a single beacon node:

```
src/initialize.py::run_services()

if vero.cli_args.enable_doppelganger_detection:
    try:
        await DoppelgangerDetector(
            beacon_chain=vero.beacon_chain,
            beacon_node=multi_beacon_node.best_beacon_node, # @audit Single beacon node
            validator_status_tracker_service=validator_status_tracker_service,
        ).detect()
    except DoppelgangersDetected:
        vero.shutdown_event.set()
        raise
```

The detector queries this single node for liveness data and determines whether any validator keys are active elsewhere. If any validator is reported as live, Vero raises `DoppelgangersDetected` and shuts down:

```
src/providers/doppelganger_detector.py::DoppelgangerDetector._fetch_liveness_data()
```

```
async def _fetch_liveness_data(
    self, epoch: int, validator_indices: list[int]
) -> list[SchemaBeaconAPI.ValidatorLiveness]:
    try:
        liveness_response = await self.beacon_node.get_liveness(
            epoch=epoch,
            validator_indices=validator_indices,
        )
    except Exception:
        self.logger.error(
            f"Failed to query beacon node for liveness data for epoch {epoch}"
            " - did you enable liveness tracking?"
        )
        raise
    else:
        self.logger.debug(f"Liveness response: {liveness_response}")
        return liveness_response.data
```

The [Beacon API specification](#) for the `/eth/v1/validator/liveness/{epoch}` endpoint explicitly acknowledges the inherent unreliability of liveness data. The specification states that liveness values returned by beacon nodes are “not canonical” and are “best-effort and based upon a subjective view of the network.” Critically, the specification warns that “a beacon node that was recently started or suffered a network partition may indicate that a validator is not live when it actually is.”

A beacon node could therefore fail to detect actual doppelganger activity through false negative liveness data. This allows Vero to proceed with validator duties whilst a doppelganger is active, creating double-attestation scenarios that result in slashing.

Proposer and attester duties are also sourced from the single best beacon node:

```
src/providers/multi_beacon_node.py::MultiBeaconNode.get_proposer_duties()
```

```
async def get_proposer_duties(
    self,
    **kwargs: Any,
) -> SchemaBeaconAPI.GetProposerDutiesResponse:
    return await self.best_beacon_node.get_proposer_duties(**kwargs) # @audit Single source
```

```
src/providers/multi_beacon_node.py::MultiBeaconNode.get_attester_duties()
```

```
async def get_attester_duties(
    self,
    **kwargs: Any,
) -> SchemaBeaconAPI.GetAttesterDutiesResponse:
    return await self.best_beacon_node.get_attester_duties(**kwargs) # @audit Single source
```

Each duty refresh overwrites local state entirely from this single response:

```
src/services/attestation.py::AttestationService._update_duties()
```

```
response = await self.multi_beacon_node.get_attester_duties(
    epoch=epoch,
    indices=_validator_indices,
)
if response.dependent_root == self.attester_duties_dependent_roots.get(epoch, None):
    # We already processed these same duties
    continue

self.attester_duties[epoch] = set() # @audit Replaced from one response
# ... snip ...
for list_of_duties in (duties_due_soon, duties_due_later):
    for duty_with_proof in await self._get_duties_with_selection_proofs(
        duties=list_of_duties,
    ):
        self.attester_duties[epoch].add(duty_with_proof)
# ... snip ...
self.attester_duties_dependent_roots[epoch] = response.dependent_root
```

If the selected beacon node returns stale or divergent data, valid duties may be temporarily suppressed, resulting in missed attestations or block proposals and reduced rewards.

This issue has a high impact as failing to detect a doppelganger may result in a validator slashing event if both Vero and another validator client are operating at the same time with the same validator keys. Additionally, stale duty data from a single node can cause missed duties and reward loss.

This issue has a low likelihood as the doppelganger scenario requires that both: (1) an actual doppelganger is operating at system startup and (2) the best beacon node is following a fork where the doppelganger seems inactive. The duty fetching scenario requires the best beacon node to return divergent duty data, which is uncommon during normal operation.

Recommendations

Implement multi-node reconciliation for doppelganger detection by querying liveness data from multiple beacon nodes and requiring agreement from a threshold of nodes before determining that a validator is not live.

For duty fetching, add optional quorum-based duty verification as a hardening mode for operators prioritising duty integrity over latency.

Add detailed logging showing which beacon nodes reported validators as live or not live, and emit explicit mismatch telemetry when duty payloads from non-selected beacon nodes diverge from the selected source.

Resolution

The development team has changed doppelganger detection to fetch liveness data from all beacon nodes. `_fetch_liveness_data()` also now raises an exception if any beacon node returns an error, and every beacon node's full liveness data is logged at debug level.

Optional quorum-based duty verification was not implemented and acknowledged with the following comment:

"The optional quorum-based duty verification recommendation was not implemented, the benefit does not seem worth the additional complexity it would require."

This issue has been resolved in commit [61d2e70](#).

VERO-03	Stale RANDAO Reveal May Lead To Invalid Block		
Assets	src/services/block_proposal.py		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

Chain reorganisations can cause validators to propose blocks with invalid RANDAO reveals, resulting in rejected blocks and missed proposal rewards.

The RANDAO reveal cache in `src/services/block_proposal.py` stores pre-computed RANDAO reveals keyed only by slot number:

```
src/services/block_proposal.py::BlockProposalService.__init__()
self.randao_reveal_cache: dict[int, str] = dict()
```

When preparing for the next slot's block proposal, the service fetches and caches the RANDAO reveal:

```
src/services/block_proposal.py::BlockProposalService._fetch_randao_reveal()

async def _fetch_randao_reveal(self, duty: SchemaBeaconAPI.ProposerDuty) -> None:
    self.logger.debug(f"Fetching RANDAO reveal for slot {duty.slot}")

    slot = int(duty.slot)
    epoch = slot // self.beacon_chain.SLOTS_PER_EPOCH

    _, randao_reveal, _ = await self.signature_provider.sign(
        message=SchemaRemoteSigner.RandaoRevealSignableMessage(
            fork_info=self.beacon_chain.get_fork_info(slot=slot),
            randao_reveal=SchemaRemoteSigner.RandaoReveal(
                epoch=str(epoch),
            ),
        ),
        identifier=duty.pubkey, # @audit Signed with specific validator's key
    )
    self.randao_reveal_cache[slot] = randao_reveal # @audit Cached by slot only
```

The RANDAO reveal is signed using the specific validator's public key, `duty.pubkey`, but the cache key includes only the slot number. When the cached value is retrieved in `_get_randao_reveal()`, no validation occurs to ensure the cached RANDAO reveal was signed by the correct validator:

```
src/services/block_proposal.py::BlockProposalService._get_randao_reveal()

async def _get_randao_reveal(
    self, slot: int, duty: SchemaBeaconAPI.ProposerDuty
) -> str:
    # Try to get it from the cache - it should be pre-populated
    # in the slot before a proposal is due
    if slot in self.randao_reveal_cache:
        return self.randao_reveal_cache[slot] # @audit No validator identity check
    # ... snip ...
```

This creates a vulnerability when chain reorganisations change proposer assignments. Consider the following scenario:

1. At slot N, `on_new_slot()` executes and checks for duties in the next slot (N+1)
2. A proposer duty exists for slot N+1 assigned to validator A (pubkey a)
3. The RANDAO reveal is fetched and signed by validator A, then cached: `randao_reveal_cache[N+1] = reveal_signed_by_A`

4. A deep chain reorganisation occurs, triggering `handle_reorg_event()` which calls `update_duties()`
5. After the reorg, proposer duties are recomputed and slot N+1 is now assigned to validator B (pubkey b)
6. When slot N+1 arrives and `_get_randao_reveal()` is called with validator B's duty, it retrieves the cached value signed by validator A
7. Validator B attempts to propose a block using validator A's RANDAO reveal
8. The block is rejected by the network because the RANDAO reveal signature verification fails

The `handle_reorg_event()` method in the base class calls `update_duties()` which refreshes the proposer duty assignments but does not invalidate the RANDAO reveal cache. Similarly, `handle_head_event()` detects duty dependent root mismatches and updates duties without clearing the cache.

Whilst deep reorganisations that change proposer assignments are uncommon, they are possible and can occur during network issues, client bugs, or attacks.

This issue has a high impact as it causes block proposals to be rejected by the network, resulting in loss of proposal rewards including both consensus rewards and any MEV value.

This issue has a low likelihood as it requires a chain reorganisation deep enough to change proposer duty assignments for upcoming slots, which is uncommon during normal network operation but possible during periods of network instability or attacks. Moreover, this must occur within the narrow time window between caching the RANDAO reveal and the next slot's proposal deadline, which further reduces the likelihood.

Recommendations

Modify the RANDAO reveal cache to key entries by both slot number and validator public key, ensuring cached values can only be used by the validator that signed them. Update the cache structure to use a tuple or composite key such as a dictionary keyed by `(slot, pubkey)` pairs.

Alternatively, implement cache invalidation logic that clears the RANDAO reveal cache whenever duties are updated due to reorganisations or dependent root mismatches. Add a cache clearing operation to the `_update_duties()` method before fetching new duties, ensuring stale cached values cannot be used after duty reassignments.

Resolution

The development team has implemented the first recommendation. The `randao_reveal_cache` type was changed from `dict[int, str]` to `dict[tuple[int, str], str]`, keying entries by `(slot, pubkey)` pairs. Both `_fetch_randao_reveal()` and `_get_randao_reveal()` were updated to accept explicit `slot` and `pubkey` parameters and use the composite cache key throughout, ensuring a cached reveal can only be retrieved by the validator that signed it.

This issue has been resolved in commit [2b1e14f](#).

VERO-04 Incorrect MEV Registration Leads To Potential Income Loss			
Assets	docs/docs/usage/mev.md docs/docs/reference/cli_reference.md src/providers/multi_beacon_node.py		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Users who follow the MEV configuration documentation may fail to have validator registrations reach their intended MEV relays, resulting in lost income from external block builders.

The `register_validator()` function in `src/providers/multi_beacon_node.py` implements the validator registration logic:

```
src/providers/multi_beacon_node.py::MultiBeaconNode.register_validator()
```

```
async def register_validator(self, **kwargs: Any) -> None:
    # Only ask one of the beacon nodes to register the validators with
    # MEV relays - no need to overwhelm them with duplicate registrations
    await self.best_beacon_node.register_validator(**kwargs)
```

The function uses the `best_beacon_node` property, which is defined in `src/providers/multi_beacon_node.py`:

```
src/providers/multi_beacon_node.py::MultiBeaconNode.best_beacon_node()
```

```
@property
def best_beacon_node(self) -> BeaconNode:
    return next(
        bn
        for bn in sorted(
            self.initialized_beacon_nodes, key=lambda bn: bn.score, reverse=True
        )
    )
```

This property returns the beacon node with the highest heuristic health score.

The MEV configuration documentation in `docs/docs/usage/mev.md` instructs users:

By default, Vero uses the first connected beacon node to submit its validator registrations (= the first URL provided in the `-beacon-node-urls` CLI argument). Therefore, ensure that the first connected beacon node is connected to all the MEV relays you wish to use.

A user following this documentation may only connect the first beacon node in their list to all desired MEV relays. However, the code does not guarantee use of the first beacon node. Instead, it always selects the highest-scoring initialised beacon node. If the first beacon node is not initialised, or its score drops below another beacon node due to network issues, slow responses, or downtime, Vero will switch to using a different beacon node for validator registrations.

If this alternative beacon node is not configured similarly, validator registrations will fail to reach MEV relays. Without successful registration, external block builders will be effectively unused. This results in validators only proposing locally-built blocks instead of potentially more profitable builder blocks, causing financial loss through missed MEV opportunities.

This issue has a medium impact as it causes moderate financial loss through missed MEV rewards whenever the beacon node scoring causes a switch away from the first beacon node.

This issue has a medium likelihood as it occurs when users follow the documentation and beacon node scores diverge, which is a plausible scenario in multi-beacon-node production environments where nodes may experience varying performance or availability.

Recommendations

Alter the function `register_validator()` so that it always attempts registration against the first beacon node, as specified in the documentation. However, if this beacon node is offline or misbehaving, registrations may be missed.

Alternatively, continue sending registrations to the highest-scoring beacon node, but add a warning in the documentation regarding this behaviour, and log a warning when the best beacon node is not the first beacon node, to alert users to potential misconfiguration.

Resolution

The development team has implemented the second alternative recommendation. The MEV documentation in `docs/docs/usage/mev.md` was corrected to accurately describe that Vero selects the highest-scoring connected beacon node for validator registrations rather than always using the first. The updated documentation advises operators to ensure all connected beacon nodes are configured to reach all intended MEV relays. A warning log message was also added to `best_beacon_node` in `src/providers/multi_beacon_node.py` that fires whenever the highest-scoring node differs from the first node in `-beacon-node-urls`, alerting operators to potential MEV misconfiguration at runtime.

This issue has been resolved in commits [4e08ecf](#) and [9504eaa](#).

VERO-05 Missing Signing Timeout Can Block All Attestation Publication			
Assets	src/services/attestation.py src/services/sync_committee.py		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

A slow or unresponsive remote signer can delay publication of all attestations for a slot, including those already signed by healthy signers, causing missed attestation rewards for all managed validators assigned to attest in that slot.

The `_get_signed_attestations()` function uses `asyncio.as_completed()` to fan-in signing responses from the remote signer. While this pattern processes results in completion order, the function only returns once every coroutine in the iterator has been awaited. No per-request timeout or overall slot-boundary deadline is applied to the signing phase:

```
src/services/attestation.py::AttestationService._get_signed_attestations()
```

```
async def _get_signed_attestations(
    self,
    slot: int,
    att_data: SchemaBeaconAPI.AttestationData,
    duties: set[SchemaBeaconAPI.AttesterDutyWithSelectionProof],
) -> list[SchemaBeaconAPI.SingleAttestation]:
    signed_attestations: list[SchemaBeaconAPI.SingleAttestation] = []

    pubkey_to_duty = {d.pubkey: d for d in duties}
    message = SchemaRemoteSigner.AttestationSignableMessage(
        fork_info=self.beacon_chain.get_fork_info(slot=slot),
        attestation=msgspec.to_builtins(att_data),
    )

    # @audit No timeout on individual awaits or on the loop as a whole
    for coro in asyncio.as_completed(
        [
            self.signature_provider.sign(
                message=message,
                identifier=duty.pubkey,
            )
            for duty in duties
        ],
    ):
        try:
            message, signature, pubkey = await coro
        except Exception as e:
            # ... snip ...
            continue

        duty = pubkey_to_duty[pubkey]
        signed_attestations.append(...)

    return signed_attestations
```

The calling function `_attest()` invokes `_get_signed_attestations()` and then `_publish_attestations()` sequentially, so publication is blocked until all signing tasks complete:

```
src/services/attestation.py:AttestationService._attest()

async def _attest(self, slot, head_event, duties):
    # ... snip ...
    att_data = await self._produce_attestation_data(slot=slot, head_event=head_event)
    # ... snip ...

    # @audit Publication blocked until all signing tasks complete
    signed_attestations = await self._get_signed_attestations(
        slot=slot, att_data=att_data, duties=duties,
    )
    await self._publish_attestations(
        slot=slot, att_data=att_data, signed_attestations=signed_attestations,
    )
```

Notably, the preceding step `_produce_attestation_data()` applies a slot-boundary timeout via `asyncio.wait_for()`, but no equivalent protection exists for the signing phase.

The `high_priority_client_session` used for attestation signing in `RemoteSigner` does not set an explicit timeout, falling back to `aiohttp's` default `ClientTimeout(total=300)` — a 5-minute total timeout per HTTP request. For attestation duties where timing is critical (attestations should ideally be published within seconds of the slot start), a single slow request can delay all attestation publication by up to 5 minutes, far exceeding the 12-second slot duration. (see [VERO-16](#))

In a multi-signer deployment via the `Keymanager`, different validator keys may be routed to different `RemoteSigner` instances. If one signer becomes unresponsive, attestations already signed by healthy signers remain unpublished until the slow request times out. This converts a partial infrastructure failure into a total attestation delay for the slot. The same pattern exists in `SyncCommitteeService`, which uses an identical `asyncio.as_completed()` loop without timeout for signing sync committee messages.

This issue has a medium impact as a slow remote signer delays publication of all attestations for the affected slot, causing missed rewards and reduced network participation for managed validators with attestation duties in that slot. The concern is limited to liveness and timing; it does not create a slashing risk.

This issue has a medium likelihood as remote signer unavailability is a realistic operational scenario. Network issues, signer overload, garbage collection pauses, or slashing protection database contention are realistic failure modes. In a multi-signer deployment, the probability of at least one signer being slow at any given time increases with the number of signers.

Recommendations

Apply a per-request timeout to each signing coroutine within the `as_completed()` loop and enforce an overall slot-boundary deadline on `_get_signed_attestations()` using `asyncio.wait_for()`. Cancel outstanding signing tasks when the deadline is reached and publish whichever attestations have been successfully signed before the cutoff. Set an explicit `ClientTimeout` on the `high_priority_client_session` in `RemoteSigner` to a value appropriate for time-sensitive duties rather than relying on the 5-minute default. Apply the same fix to the equivalent pattern in `SyncCommitteeService`.

Resolution

The development team has added a shared helper to collect signing results into fast and slow batches, so that slow responders do not block publication of fast ones. `AttestationService` and `SyncCommitteeService` were updated to use this helper. Explicit session timeouts were also set on both `RemoteSigner` client sessions, replacing the 5-minute default with 5 seconds for the high-priority session used by time-sensitive duties and 10 seconds for the low-priority session.

This issue has been resolved in commits [52b9b69](#) and [4ad8127](#).

VERO-06 Strict Slot Equality May Drop Valid Attestations			
Assets	src/services/attestation.py src/providers/beacon_chain.py		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

`AttestationService.attest_if_not_yet_attested()` requires the requested duty slot to exactly match the locally computed `current_slot`, which can reject otherwise valid attestations when the host clock is slightly skewed or near a slot-boundary transition.

In `AttestationService.handle_head_event()`, Vero forwards the beacon-node head event slot directly into `attest_if_not_yet_attested()`. The latter then hard-fails unless `slot == self.beacon_chain.current_slot`. However, `BeaconChain.current_slot` is derived from local wall-clock time, not from beacon-node slot observations. This creates a liveness hazard: a head event for the effective network slot can be rejected as “invalid” whenever local time differs sufficiently from network time or is quantised across a boundary.

```
src/services/attestation.py::AttestationService.handle_head_event()
```

```
async def handle_head_event(
    self, event: SchemaBeaconAPI.HeadEvent, beacon_node_host: str
) -> None:
    # ... snip ...
    await self.attest_if_not_yet_attested(slot=int(event.slot), head_event=event)
```

```
src/services/attestation.py::AttestationService.attest_if_not_yet_attested()
```

```
async def attest_if_not_yet_attested(
    self,
    slot: int,
    head_event: SchemaBeaconAPI.HeadEvent | None = None,
) -> None:
    # ... snip ...
    if slot != self.beacon_chain.current_slot:
        raise RuntimeError( # @audit strict equality rejects near-boundary/skewed valid duties
            f"Invalid slot for attestation: {slot}. Current slot: {self.beacon_chain.current_slot}"
        )
```

```
src/providers/beacon_chain.py::BeaconChain.current_slot()
```

```
@property
def current_slot(self) -> int:
    seconds_elapsed = floor(time.time()) - self.genesis_time # @audit local wall clock
    seconds_elapsed = max(0, seconds_elapsed)
    return seconds_elapsed // self.SECONDS_PER_SLOT
```

This issue has a medium impact as it can degrade validator liveness and rewards through missed attestations and avoidable penalties, and can repeatedly affect duty performance across ongoing operation.

This issue has a medium likelihood as small clock skew, slot-boundary timing jitter, or transient time-synchronisation drift are realistic in production environments, and the implementation currently provides no tolerance window.

Recommendations

Replace strict slot equality with a bounded acceptance window and explicit anti-duplication checks. Permit attestation attempts for a narrow set such as `{current_slot - 1, current_slot, current_slot + 1}`, while preserving slash-safety by tracking per-validator/per-duty signing state. Gate `current_slot + 1` behind a strict boundary-time tolerance and reject clearly premature future duties. Add metrics and alerts for sustained local-versus-network slot divergence so operational teams can correct host time synchronisation quickly.

Resolution

The development team has moved the strict slot equality check before the fallback scheduled attestation job is removed. This allows Vero to attest at 1/3rd into the slot when the local clock is skewed and the head event attestation fails due to the difference in slot.

Regarding the recommendation to add metrics and alerts for slot divergence, the development team noted:

"In terms of metrics indicating divergence, a slot mismatch will show up in the `errors` metric in the `OTHER` error_type category. This can already be alerted on. Adding this as a separate metric is currently not planned."

This issue has been resolved in commit [4ada6a9](#).

VERO-07 Non-Atomic Duty Refresh Can Cause Missed Attestations			
Assets	src/services/attestation.py src/services/validator_duty_service.py		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A race condition in `_update_duties()` can cause validators to silently miss attestation duties during chain reorganisations, resulting in lost attestation rewards.

The `_update_duties()` method in `AttestationService` refreshes attester duties when a dependent root change is detected, such as following a chain reorganisation. The method first clears all duties for the affected epoch by assigning an empty set to `self.attester_duties[epoch]`, then asynchronously repopulates them by fetching selection proofs via `_get_duties_with_selection_proofs()`. The signing operation involves remote signer I/O through `signature_provider.sign_in_batches()`, which yields control back to the event loop:

```
src/services/attestation.py::AttestationService._update_duties()

# @audit Duties are cleared immediately before any new duties are fetched
self.attester_duties[epoch] = set()

# ... snip ...

for list_of_duties in (duties_due_soon, duties_due_later):
    # @audit Yields to event loop during remote signing I/O
    for duty_with_proof in await self._get_duties_with_selection_proofs(
        duties=list_of_duties,
    ):
        self.attester_duties[epoch].add(duty_with_proof)
```

Between the assignment of the empty set and the completion of the `await`, `self.attester_duties[epoch]` is either empty or only partially populated. The `_update_duties_lock` (defined in `ValidatorDutyService`) only prevents concurrent `_update_duties()` invocations; it does not guard `self.attester_duties` against concurrent reads from the attestation execution path. The `_get_duties_for_slot()` method reads directly from `self.attester_duties[epoch]` without acquiring any lock:

```
src/services/attestation.py::AttestationService._get_duties_for_slot()

def _get_duties_for_slot(self, slot):
    epoch = slot // self.beacon_chain.SLOTS_PER_EPOCH
    slot_attester_duties = {
        # @audit Reads from potentially empty set during duty refresh window
        duty for duty in self.attester_duties[epoch] if int(duty.slot) == slot
    }
    for duty in slot_attester_duties:
        self.attester_duties[epoch].remove(duty)
    return slot_attester_duties
```

The following scenario demonstrates the race condition:

1. A chain reorganisation occurs, causing a dependent root change for epoch E.
2. `handle_head_event()` or `handle_reorg_event()` schedules `_update_duties()` as a background task via `task_manager.create_task()`.
3. `_update_duties()` detects the dependent root mismatch and clears `self.attester_duties[E] = set()`.

4. `_update_duties()` calls `await self._get_duties_with_selection_proofs()`, yielding to the event loop. This call performs remote signing I/O, which can be slow, particularly for large validator sets. The codebase itself acknowledges this: “For large amounts of validators, the `_get_duties_with_selection_proofs` can take quite a while.”
5. During the `await`, a head event arrives for slot `S`. The event handler calls `attest_if_not_yet_attested(slot=S)`.
6. `_get_duties_for_slot(S)` reads `self.attester_duties[E]` and finds an empty set.
7. `attest_if_not_yet_attested()` finds no duties and returns without producing an attestation for slot `S`. If called from a head event, the backup attestation deadline timer for slot `S` is also cancelled. No reliable mechanism re-triggers attestation for that slot after the refresh completes, so the attestation is effectively lost.

The `duties_due_soon` prioritisation (processing imminent duties before later ones) partially reduces the window for the most imminent slot, but the entire set is cleared before any duties are added back, so an interleaving attestation read still observes the empty state.

This issue has a medium impact as affected validators miss attestation rewards for the slot(s) whose attestation window coincides with the signing operation. For large validator sets, all validators with duties in the epoch can be simultaneously affected. The impact is limited to missed rewards; this is not a slashable offence, and duties are correct once the refresh completes.

This issue has a low likelihood as it requires a chain reorganisation with a changed dependent root and an attestation opportunity (head event or scheduled deadline) arriving during the signing I/O window. However, the probability increases with higher validator count (longer signing extends the window), with the number of connected beacon nodes (more head events arriving during the window), and during periods of network instability, which is precisely when attestation correctness matters most.

Recommendations

Build the refreshed duty set in a temporary variable and replace `self.attester_duties[epoch]` with a single atomic assignment after all duties have been successfully fetched and their selection proofs obtained. This ensures no other coroutine can observe an empty or partially populated duty set during the refresh window.

Resolution

The development team has implemented the recommendation. Duties are now accumulated into a local `refreshed_duties` variable and assigned to `self.attester_duties[epoch]` atomically.

This issue has been resolved in commit [511d9a2](#).

VERO-08	Overly Permissive Permissions On Keymanager API Token File		
Assets	src/api/keymanager/api.py		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Vero writes the Keymanager API bearer token using default file-creation permissions, which can expose the token to other local users and allow unauthorised key-management requests.

In `src/api/keymanager/api.py`, the function `_get_bearer_token_value()` creates `keymanager-api-token.txt` with `Path.open(fp, "w")`. On Unix systems, this uses mode `00666` masked by process `umask`, which is commonly `00022`, producing a world-readable file (`00644`). The same token then authorises sensitive Keymanager API operations.

```
src/api/keymanager/api.py::_get_bearer_token_value()

def _get_bearer_token_value(cli_args: CLIArgs) -> str:
    fp = cli_args.keymanager_api_token_file_path
    if fp.exists():
        with Path.open(fp) as f:
            return f.read().strip()

    bearer_token_value = os.urandom(32).hex()
    with Path.open(fp, "w") as f: # @audit file mode derives from umask, often world-readable
        f.write(bearer_token_value)

    return bearer_token_value
```

An attacker with local unprivileged access to the host can read the token and use it in the `Authorization: Bearer ...` header to call Keymanager API endpoints on loopback, including operations that alter validator configuration or disrupt duties.

This issue has a medium impact as token disclosure enables unauthorised authenticated control over validator Keymanager API actions, which can cause operational disruption and financial downside.

This issue has a low likelihood as exploitation generally requires local host access or a shared runtime environment with multiple users.

Recommendations

Create the token file with owner-only permissions (`00600`) and enforce strict permissions when loading an existing token file. Use an atomic create path (for example, `os.open(..., O_CREAT | O_EXCL, 00600)`), then `chmod(00600)` and fail startup if the file remains group- or world-readable.

Resolution

The development team has implemented the recommendation. Permissions are now set to `00600` when creating the token file, and an explicit check was added to ensure that permissions are set to `00600` on startup.

This issue has been resolved in commit [84b9d6e](#).

VERO-09	Retired RemoteSigner Instances Are Never Closed		
Assets	src/providers/keymanager.py src/providers/remote_signer.py src/api/keymanager/remote_key_manager_endpoints.py		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The Keymanager remote-key lifecycle leaks live `RemoteSigner` instances, allowing authenticated API-driven key churn to accumulate background tasks and HTTP client sessions until service degradation or process-level resource exhaustion occurs.

In `src/providers/keymanager.py`, `_update_pubkey_to_remote_signer_mapping()` rebuilds `pubkey_to_remote_signer` from database rows and creates new signers through `AsyncExitStack.enter_async_context()`. However, when keys are deleted or remapped to new URLs, signers no longer referenced by the new mapping are not explicitly retired; they remain registered in the long-lived exit stack until global shutdown.

```
src/providers/keymanager.py::Keymanager._update_pubkey_to_remote_signer_mapping()
async def _update_pubkey_to_remote_signer_mapping(self) -> None:
    rows = self.db.fetch_all("SELECT pubkey, url FROM keymanager_data;")
    signers_by_url = {
        signer.url: signer for signer in self.pubkey_to_remote_signer.values()
    }
    new_mapping = {}

    for pubkey, url in rows:
        # ... snip ...
        signer = await self._exit_stack.enter_async_context(
            RemoteSigner(url, vero=self.vero, process_pool_executor=self.process_pool_executor)
        ) # @audit new signer is registered for close-on-global-exit only
        signers_by_url[url] = signer
        new_mapping[pubkey] = signer

    self.pubkey_to_remote_signer = new_mapping # @audit dropped mapping refs are not closed
```

This mapping refresh path is triggered by `import_remote_keys()` and `delete_remote_keys()`, which are reachable through authenticated Keymanager endpoints (POST `/eth/v1/remotekeys` and DELETE `/eth/v1/remotekeys`). An attacker with bearer token access can repeatedly import and delete keys with rotating URLs, causing unbounded signer accumulation at runtime.

Each leaked signer starts `poll_health_periodically()` and allocates two `aiohttp.ClientSession` objects in the method `__aenter__()`. These resources are only released in `RemoteSigner.__aexit__()`, which is not called for retired signers during normal mapping updates.


```
src/providers/remote_signer.py::RemoteSigner
```

```
class RemoteSigner(SignatureProvider):
    # ... snip ...

    def __init__(
        # ... params ...
    ):
        # ... snip ...
        # Regularly poll the health status of the remote signer
        # @audit unbounded periodic task
        if vero is not None:
            vero.task_manager.create_task(self.poll_health_periodically())
        # ... snip ...

    async def __aenter__(self) -> Self:
        # ... snip ...
        # @audit starts two client sessions for the signer on entry
        self.low_priority_client_session = aiohttp.ClientSession(
            # ... snip ...
        )

        self.high_priority_client_session = aiohttp.ClientSession(
            # ... snip ...
        )
        # ... snip ...

    async def __aexit__(
        # ... params ...
    ) -> None:
        # @audit sessions are closed only when this signer exits
        for session in (
            self.low_priority_client_session,
            self.high_priority_client_session,
        ):
            if not session.closed:
                await session.close()
        if self.process_pool_executor is not None:
            self.process_pool_executor.shutdown(wait=False, cancel_futures=True)
```

This issue has a medium impact as sustained churn can gradually consume file descriptors, memory, and scheduler capacity, degrading signing reliability over time, potentially causing missed duties. Material operational impact generally requires many remote signer instances.

This issue has a low likelihood as exploitation requires access to the Keymanager API bearer token. The token is stored locally and file permissions are not hardened to `006000`, so depending on host `umask` the file can be group- or world-readable (see [VERO-08](#)). Accidental triggering through normal validator churn is also unlikely because the leak granularity is per unique remote signer URL, not per key. `_update_pubkey_to_remote_signer_mapping()` reuses existing signers when the URL matches, so bulk key operations against the same remote signer orphan at most one instance. Reaching meaningful resource pressure would require cycling through many distinct URLs or repeatedly deleting and re-importing keys across many mapping rebuilds.

Recommendations

Implement explicit signer retirement during `_update_pubkey_to_remote_signer_mapping()` by computing the set difference between previously active signer instances and newly active signer instances, then closing every retired signer immediately.

Track health-poll tasks per signer and cancel them on signer retirement before releasing network resources. Keep signer ownership scoped to active URL membership rather than process lifetime.

Decouple shared executor ownership from per-signer lifetime so that closing a retired signer does not shut down executor resources still needed by active signers.

Resolution

The development team has implemented the recommendation. Vero now retires `RemoteSigner` instances when they are removed from active use. A closed `RemoteSigner` instance will close its client sessions and cancel its health poll task promptly.

This issue has been resolved in commits [f9287f9](#) and [9981a10](#).

VERO-10 Single Beacon Node Can Trigger False Slashing Halt			
Assets	src/services/event_consumer.py src/providers/multi_beacon_node.py src/services/validator_status_tracker.py src/services/attestation.py src/services/block_proposal.py src/initialize.py		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A single malfunctioning beacon node can trigger a false slashing detection through two distinct vectors, causing Vero to halt attestation and block-proposal duties until manual intervention occurs. This results in continuous missed attestations and block proposals, leading to financial losses through missed rewards and inactivity penalties.

Vero relies on single-beacon-node trust in two critical code paths for slashing detection. First, when beacon nodes emit slashing events via the event stream, Vero accepts these events from any single node without cross-validation. Vero subscribes to event streams from all configured beacon nodes as shown in `src/initialize.py`:

```
src/initialize.py::run_services()

event_consumer_service = EventConsumerService(
    beacon_nodes=multi_beacon_node.beacon_nodes, # @audit Subscribes to all beacon nodes
    vero=vero,
)
```

Second, when querying validator statuses, Vero uses a first-responder pattern that accepts the first successful response. The `MultiBeaconNode.get_validators()` method calls `_get_first_beacon_node_response()`, which trusts whichever beacon node responds fastest:

```
src/providers/multi_beacon_node.py::MultiBeaconNode._get_first_beacon_node_response()

async def _get_first_beacon_node_response(
    self,
    func_name: str,
    **kwargs: Any,
) -> Any:
    tasks = [
        asyncio.create_task(getattr(bn, func_name)(**kwargs))
        for bn in self.initialized_beacon_nodes
    ]

    for coro in asyncio.as_completed(tasks):
        try:
            resp = await coro
        except Exception:
            continue
        else:
            # @audit First successful response is trusted.
            for task in tasks:
                task.cancel()
            return resp
```

Both paths converge on the same mechanism in `ValidatorStatusTrackerService`. When slashing is detected through either an event handler or status query, the global `slashing_detected` flag is set to `True`. Once this flag is set, all attestation and block proposal operations immediately halt:

```
src/services/attestation.py::AttestationService.attest_if_not_yet_attested()
if (
    self.validator_status_tracker_service.slashing_detected
    and not self.cli_args.disable_slashing_detection
):
    raise RuntimeError("Slashing detected, not attesting")
```

A malfunctioning beacon node could produce false slashing signals due to software bugs, corrupted state, database inconsistencies, or malicious behaviour. Through the event stream path, a beacon node might emit spurious slashing events for validators it incorrectly believes have been slashed. Through the status query path, a beacon node might return slashed status for active validators due to state corruption or being on a wrong fork. In both cases, Vero accepts the single beacon node's view without requiring confirmation from other beacon nodes.

This issue has a medium impact as it causes complete cessation of attestation and block-proposal duties, resulting in continuous missed attestations and block proposals, leading to financial losses through missed rewards and inactivity penalties. The validator remains halted until manual intervention occurs. However, it does not cause slashing or compromise validator keys, and normal operations can resume after resolving the malfunctioning beacon node.

This issue has a low likelihood as it requires a beacon node to malfunction in specific ways that produce false slashing signals, whether through spurious events or incorrect status responses. Whilst not a common failure mode, it is possible given the complexity of consensus client implementations and the variety of failure scenarios that can affect beacon node state.

Recommendations

Implement a consensus mechanism for slashing detection that requires confirmation from multiple beacon nodes before halting validator operations. This should apply to both attack vectors identified above.

For slashing events received via the event stream, query the validator status from a threshold of beacon nodes to verify whether the validators in question are actually marked as slashed in their state. Set the `slashing_detected` flag to `True` if a quorum of beacon nodes confirms the slashing status.

For validator status queries, replace the first-responder pattern with a threshold-based approach for safety-critical status updates, especially when checking for `SLASHED_STATUSES`. Require quorum agreement before setting the global halt flag. When beacon nodes disagree on validator statuses, retain the previous trusted snapshot and retry with backoff rather than trusting a single potentially incorrect response.

Anti-slashing bias can be enforced by temporarily setting the `slashing_detected` flag whilst the beacon node consensus mechanism is consulted, ensuring protection against slashing at the cost of halting operations during the verification process.

Add explicit telemetry and logging for cross-node disagreements on slashing status, including which beacon nodes reported which states. This helps operators identify malfunctioning beacon nodes and understand the decision-making process when slashing is detected.

Resolution

The development team has acknowledged the issue with the following comment:

"This works as intended. We intentionally prefer Vero to err on the side of caution in a situation like this and halt even if only a single beacon node reports one of its managed validators got slashed."

VERO-11 Unbounded Provider Response Reads Can Exhaust Memory			
Assets	src/providers/beacon_node.py src/providers/remote_signer.py		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Multiple provider paths read remote responses without enforcing strict payload limits, allowing a malicious or misbehaving upstream service to trigger memory exhaustion and disrupt validator operations.

In the methods `BeaconNode._handle_nok_status_code()` and `BeaconNode._make_request()`, the implementation calls `response.text()` for both error and success paths. In `RemoteSigner`, `get_public_keys()` and `sign()` also call `resp.text()` on non-OK responses, while `sign()` and `get_healthcheck_status()` call `resp.read()` and decode the full body. The configured `read_bufsize=2**19` only controls per-chunk buffering and does not cap total response bytes.

`BeaconNode.subscribe_to_events()` has a related issue in the SSE stream parser: it appends event lines into `event_data` until a terminator line is observed, with no maximum event size enforced and `sock_read=None`. A beacon node can keep sending event data lines without terminating the event, causing unbounded accumulation in memory.

```
src/providers/beacon_node.py::BeaconNode._handle_nok_status_code()

@staticmethod
async def _handle_nok_status_code(response: aiohttp.ClientResponse) -> None:
    if response.ok:
        return

    resp_text = await response.text() # @audit unbounded full-body read on error path
    # ... snip ...
```

```
src/providers/beacon_node.py::BeaconNode._make_request()

async def _make_request(...):
    # ... snip ...
    async with self.client_session.request(...) as resp:
        await self._handle_nok_status_code(response=resp)
    # ... snip ...
    return await resp.text() # @audit unbounded full-body read on success path
```

```
src/providers/remote_signer.py::RemoteSigner.sign()

async with self._get_session_for_message(message).post(
    _endpoint.format(identifier=identifier),
    # ... snip ...
) as resp:
    if not resp.ok:
        raise ValueError(
            # @audit unbounded full-body read on error path
            f"NOK status code received ({resp.status}) from remote signer: {await resp.text()}",
        )
    # ... snip ...
    # @audit unbounded full-body read on success path
    return message, (await resp.read()).decode(), identifier
```

```
src/providers/beacon_node.py::BeaconNode.subscribe_to_events()

async def subscribe_to_events(...):
    # ... snip ...
    event_data = []
    next_line = (await anext(events_iter)).decode()
    while next_line not in ("\n", "\r\n"):
        event_data.append(next_line) # @audit unbounded event accumulation
        next_line = (await anext(events_iter)).decode()
```

This issue has a medium impact as successful exploitation can crash or destabilise the validator client, leading to missed duties and operational penalties across beacon-node and remote-signer integrations.

This issue has a low likelihood as exploitation generally requires control of, or severe malfunction in, a configured beacon-node or remote-signer endpoint.

Recommendations

Implement strict maximum response-size limits for all provider reads. Stream HTTP bodies in bounded chunks, track cumulative bytes, and abort requests once limits are exceeded. Add explicit maximum event-size limits for `subscribe_to_events()` and close the stream when a single event crosses the configured threshold. Apply tighter endpoint-specific limits where expected payload sizes are known, and avoid including full oversized response bodies in raised exceptions.

Resolution

The development team has added bounded response handling based on the `Content-Length` header. While this does not protect against malicious API servers that could trivially spoof the header, the team considers this an acceptable trade-off given their trust model, as noted in the following comment:

"Vero's trust model assumes the API servers it is connected to (beacon nodes, remote signers) do not intentionally return maliciously formatted responses. An API server reporting its response size as X but actually responding with much more than X bytes is not something we reasonably expect to happen unintentionally. Given this, we have chosen to partially protect Vero from large API responses, trusting the response size as reported by the API server."

This issue has been resolved in commit [0101850](#).

VERO-12 Keymanager API Returns Non-Compliant Error Status Codes			
Assets	src/api/keymanager/middlewares.py src/providers/keymanager.py src/api/keymanager/remote_key_manager_endpoints.py src/api/keymanager/voluntary_exit_endpoints.py src/api/keymanager/graffiti_endpoints.py src/spec/utils.py src/schemas/keymanager_api.py		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The Keymanager API error-handling middleware maps several client-error conditions to HTTP status code 500 Internal Server Error, where the specification requires status code 404 Not Found or 400 Bad Request responses, causing operator integrations built against the standard to misclassify failures.

The central exception handler in `src/api/keymanager/middlewares.py` only maps `msgspec.ValidationError` to HTTP status code 400; all other exceptions, including `PubkeyNotFound`, `msgspec.DecodeError`, and `ValueError`, resolve to HTTP status code 500.

```
src/api/keymanager/middlewares.py::exception_handler()
except msgspec.ValidationError as e:
    status_code = 400
    msg = repr(e)
except PubkeyNotFound as e:
    status_code = 500 # @audit spec requires 404 for unknown pubkeys
    msg = repr(e)
except Exception as e:
    status_code = 500 # @audit DecodeError and ValueError fall through here
    msg = repr(e)
```

This produces three classes of spec deviation:

1. `PubkeyNotFound` returns status code 500 instead of 404 across the `gas_limit`, `feerecipient`, `graffiti`, and `voluntary_exit` endpoint groups. The specification defines 404 for unknown-pubkey conditions on all of these endpoints.
2. `msgspec.DecodeError`, raised when request bodies contain malformed JSON, is not caught by the `ValidationError` branch and falls through to the generic `Exception` handler, returning status code 500 instead of 400.

```
src/api/keymanager/remote_key_manager_endpoints.py::remote_keys_post()
request_data = msgspec.json.decode(
    await request.content.read(),
    type=SchemaKeymanagerAPI.ImportRemoteKeysRequest, # @audit malformed JSON raises DecodeError, not ValidationError
)
```

3. `ValueError` exceptions from input validation are also caught by the generic branch, returning status code 500 instead of 400. This affects overlong graffiti values rejected by `encode_graffiti()`, malformed epoch query parameters parsed via bare `int()` in the voluntary exit endpoint, and inactive validator state lookups. Negative epoch values are also not explicitly rejected at the endpoint boundary.

```
src/api/keymanager/voluntary_exit_endpoints.py::voluntary_exit_post()
epoch=int(epoch) if epoch else None, # @audit malformed epoch raises ValueError; no non-negative check
```

Additionally, path `pubkey` values are consumed as raw strings from `request.match_info` without validation against the `^0x[a-fA-F0-9]{96}$` format, so invalid-format values propagate to the provider layer and return status code `500` through the not-found path instead of status code `400`.

Existing integration tests already assert `500` for several of these cases, confirming these behaviours are intentionally implemented rather than incidental.

This issue has a low impact as specification-deviating responses can cause operator integrations to mishandle errors or execute incorrect fallback logic, but authentication and core signing boundaries remain intact.

This issue has a low likelihood as these mismatches are concentrated in error-path handling, while normal successful API flows return the expected responses.

Recommendations

Map `PubkeyNotFound` to HTTP status code `404`, catch `msgspec.DecodeError` alongside `ValidationError` and map to HTTP status code `400`, and add explicit `ValueError` handling for input validation failures as HTTP status code `400`. Validate path `pubkey` format and `epoch` query values at the endpoint boundary before provider calls.

Resolution

The development team has addressed all three classes of spec deviation. The exception handler in `middlewares.py` now maps `PubkeyNotFound` to HTTP status code `404`, and catches `msgspec.DecodeError` and `ValueError` alongside `msgspec.ValidationError` to return HTTP status code `400`. A new `path_param_validation` middleware validates path `pubkey` parameters against the `^0x[a-fA-F0-9]{96}$` format before reaching endpoint handlers, and the voluntary exit endpoint now explicitly rejects negative `epoch` values.

The development team also noted that the Keymanager API specification itself contained inconsistencies around `404` semantics on some endpoints, which has since been corrected upstream via [ethereum/keymanager-APIs PR #85](#).

This issue has been resolved in commit [1afe756](#).

VERO-13	Stale MEV Relay Registration Before Block Proposal		
Assets	src/services/block_proposal.py		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

External builder network relay registration may not occur before a validator proposes a block, potentially making the block builder network ineffective or, if a registration already exists, causing stale fee recipient and gas limit preferences to be used for that proposal.

The `register_validators()` function spreads relay registrations across an epoch using round-robin partitioning. In contrast, `prepare_beacon_proposer()` is explicitly re-called in the slot immediately preceding a detected block proposal duty, which the inline comment acknowledges is required to ensure an up-to-date fee recipient is set.

```
src/services/block_proposal.py::BlockProposalService.on_new_slot()

duty_for_next_slot = self.duty_for_slot(slot + 1)
if duty_for_next_slot:
    await self._fetch_randao_reveal(duty=duty_for_next_slot)
    # Call the `prepare_beacon_proposer` endpoint one more time
    # just before a block proposal is scheduled to decrease
    # the chances of the fee recipient being set incorrectly,
    # e.g., due to a beacon node restarting.
    await self.prepare_beacon_proposer()

if self.cli_args.use_external_builder:
    # @audit register_validators runs as a background task for the current slot's
    # round-robin bucket – not specifically for the next-slot proposer
    self.task_manager.create_task(self.register_validators(current_slot=slot))
```

The `register_validators()` call registers only those validators whose index satisfies the round-robin condition for the current slot:

```
src/services/block_proposal.py::BlockProposalService.register_validators()

validators_to_register = [
    v
    for v in active_and_pending_validators
    if v.index % slots_per_epoch == current_slot % slots_per_epoch
]
```

The two functions serve distinct roles: `prepare_beacon_proposer()` updates the beacon node's internal fee recipient cache, which is used only when constructing local execution payloads; `register_validator()` informs external builders of the validator's preferred fee recipient and gas limit, which builders use when constructing a block.

If an operator updates `fee_recipient` or `gas_limit` via the Keymanager API, and that validator has a block proposal duty before the round-robin cycle reaches its index again, the external block builders will hold stale preferences. Under normal operation the stale window is bounded by at most one epoch.

A separate gap exists at startup. `register_validators()` is not called during service initialisation: `BlockProposalService.__aenter__()` calls `prepare_beacon_proposer()` but not `register_validators()`. The first registration is deferred to the first `on_new_slot()` invocation, and even then it runs as a background task after `propose_block()` has already executed, and only a subset of validators are registered based on the round-robin condition. On a fresh deployment where the relay holds no prior registration for these validators, a block proposal could occur with the builder network relay entirely unaware of the validator's preferences.

Both per-validator overrides are affected:

```
src/services/block_proposal.py::BlockProposalService.register_validators()

fee_recipient=default_fee_recipient
if not self.keymanager.enabled
else self.keymanager.pubkey_to_fee_recipient_override.get(
    v.pubkey, default_fee_recipient
),
gas_limit=default_gas_limit
if not self.keymanager.enabled
else self.keymanager.pubkey_to_gas_limit_override.get(
    v.pubkey, default_gas_limit
),
```

This issue has a low impact. A stale registration may cause MEV and block proposal rewards to be sent to the old address for the affected proposal. If the operator still controls that address the funds are recoverable; if the fee recipient was changed in response to a security incident and the old address is no longer accessible, the MEV rewards are lost. If a registration is entirely missing, the builder network may be unable to construct a block for that proposal, resulting in fallback to a local block building path without the benefit of MEV value capture.

This issue has a low likelihood. It requires the Keymanager API to be enabled, a fee recipient or gas limit preference to be changed recently, and a proposal duty to coincide with the stale window before the round-robin re-registers the affected validator. The startup pathway requires no such preconditions but is limited to fresh deployments where the relay has no prior registration, and a proposal duty must fall before `register_validators()` has executed for the validator which is proposing a block.

Recommendations

Ensure that a proposing validator is always included in the `register_validators()` call before its duty is scheduled, mirroring the existing defensive pattern used for `prepare_beacon_proposer()`. In `on_new_slot()`, add a targeted registration for the next-slot proposer when external builder use is enabled, so that the MEV relay always receives up-to-date fee recipient and gas limit preferences before block production.

Resolution

The development team has added a targeted `register_validators()` call for the next slot proposer alongside the existing `prepare_beacon_proposer()` call in `on_new_slot()` when a proposal duty is detected. The `register_validators()` method now accepts an optional `pubkeys_to_register` parameter to target specific validators, while retaining the round-robin partitioning for the regular background registration path. An early return guard on `use_external_builder` was also moved into `register_validators()` itself, allowing the caller to invoke it unconditionally.

This issue has been resolved in commit [d9e483d](#).

VERO-14 Head Event Dependent Root Checks Are Not Epoch-Specific

Assets `src/services/attestation.py`
`src/services/block_proposal.py`

Status **Closed:** See [Resolution](#)

Rating **Informational**

Description

The dependent root check in `AttestationService.handle_head_event()` deviates from the Beacon API specification by not associating each dependent root with its corresponding epoch.

The Beacon API specifies that when processing head events, validators should verify the dependent root for each duty epoch against the corresponding field in the head event. Because attester duty dependent roots use the formula `get_block_root_at_slot(state, compute_start_slot_at_epoch(epoch - 1) - 1)`, the correct mapping for a head event in epoch E is:

- Attester duties for epoch E: compare stored dependent root against `event.previous_duty_dependent_root`
- Attester duties for epoch E+1: compare stored dependent root against `event.current_duty_dependent_root`

In `AttestationService.handle_head_event()`, both roots are instead checked against all stored dependent root values across all epochs:

```
src/services/attestation.py::AttestationService.handle_head_event()
if any(
    # @audit .values() searches across ALL epochs, not the specific epoch each root corresponds to
    root not in self.attester_duties_dependent_roots.values()
    for root in (
        event.previous_duty_dependent_root,
        event.current_duty_dependent_root,
    )
):
```

Since `attester_duties_dependent_roots` is a `dict[int, str]` mapping epoch to dependent root, calling `.values()` discards the epoch association. A root that has changed for one epoch could incorrectly match a stored root from a different epoch, causing the check to pass without triggering a duty refresh.

In practice, this scenario is cryptographically infeasible because dependent roots are block hashes at distinct slot boundaries (`compute_start_slot_at_epoch(epoch - 1) - 1` vs `compute_start_slot_at_epoch(epoch) - 1`), and a cross-epoch hash collision cannot occur. The code is therefore functionally correct despite not expressing the intended per-epoch semantics.

The method `BlockProposalService.handle_head_event()` has the same `.values()` pattern. It checks the event's `current_duty_dependent_root` against all stored proposer duty dependent roots rather than the specific epoch:

```
src/services/block_proposal.py::BlockProposalService.handle_head_event()
if (
    # @audit .values() searches across ALL epochs, not the specific epoch
    event.current_duty_dependent_root
    not in self.proposer_duties_dependent_roots.values()
):
```

This issue is rated as informational as the code is functionally correct in all realistic scenarios due to the cryptographic properties of block hashes. The finding relates to spec compliance and code clarity rather than a practical failure mode.

Recommendations

Perform epoch-specific dependent root comparisons in both `AttestationService.handle_head_event()` and `BlockProposalService.handle_head_event()`.

Compute the head event's epoch from `event.slot` and compare the relevant dependent root field against the stored root for the corresponding epoch rather than searching across all stored values.

For the attestation service, compare `event.previous_duty_dependent_root` against the stored root for the head event's epoch and `event.current_duty_dependent_root` against the stored root for the following epoch.

For the block proposal service, compare `event.current_duty_dependent_root` against the stored root for the head event's epoch. This would align the code with the Beacon API specification and make the intended semantics explicit.

Resolution

The development team has acknowledged the issue with the following comment:

"A fix was prepared but testing revealed inconsistent dependent roots being returned by some CL client implementations, triggering unnecessary duty re-fetching that was not present before. These duty dependent roots have been an issue in the past, as it is very easy to get them wrong through off-by-one errors.

We believe it would be good to apply this fix once we're closer to the Gloas fork, at which point we can test pre-Gloas and post-Gloas behaviour across all clients and ensure they all match."

VERO-15 Pre-Emptive Duty Consumption Can Cause Irrecoverable Slot Loss On Proposal Failure	
Assets	src/services/block_proposal.py src/providers/multi_beacon_node.py
Status	Closed: See Resolution
Rating	Informational

Description

The proposal duty is consumed before the block is successfully published, meaning a downstream failure results in an irrecoverable missed slot with no retry path.

In `propose_block()`, the selected duty is removed from `self.proposer_duties[epoch]` before `_propose_block()` executes. If the subsequent call fails at any stage — block decode, signing, or publishing — the duty is already gone and the service cannot retry within the same slot.

The risk is compounded downstream. `_produce_best_block()` selects the candidate with the highest reported value before full parse validation, and `publish_block_v2()` treats the operation as successful when any single beacon node responds. A malformed high-value response can therefore consume the only proposal attempt.

```
src/services/block_proposal.py::BlockProposalService.propose_block()
```

```
duty = self.duty_for_slot(slot=slot)
if duty is None:
    self.logger.debug(f"No remaining proposer duties for slot {slot}")
    return

epoch = slot // self.beacon_chain.SLOTS_PER_EPOCH
self.proposer_duties[epoch].remove(duty)

try:
    await self._propose_block(slot=slot, duty=duty)
finally:
    self._last_slot_duty_completed_for = slot
```

```
src/providers/multi_beacon_node.py::MultiBeaconNode._produce_best_block()
```

```
if block_value > best_block_value:
    best_block_value = block_value
    best_block_response = response

# ...

return self._parse_block_response(
    response=best_block_response,
), best_block_response
```

```
src/providers/multi_beacon_node.py::MultiBeaconNode._get_all_beacon_node_responses()
responses: list[Any] = []
for res in await asyncio.gather(
    *[getattr(bn, func_name)(**kwargs) for bn in beacon_nodes_to_use],
    return_exceptions=True,
):
    if isinstance(res, Exception):
        continue
    responses.append(res)

if len(responses) == 0:
    raise RuntimeError(
        f"Failed to get a response from all beacon nodes for {func_name}",
    )
```

This issue is rated as informational because the failure mode is narrow and requires a specific sequence of downstream errors in the same slot after preemptive duty removal. The primary consequence is occasional missed proposer rewards rather than a broader liveness or safety failure.

Recommendations

Defer duty removal until publish success, or reinsert the duty on recoverable failure while the slot is still valid. Also validate candidate blocks before final best-value selection, or keep fallback ordering so one malformed high-value response cannot consume the only proposal attempt.

Resolution

The development team has acknowledged the issue with the following comment:

"While we agree there is some room for improvement, we have looked into the implementation and found it requires additional complexity that does not seem worth the small benefit. Ultimately the issue described here is triggered by misbehaving beacon nodes rather than faulty behaviour within Vero itself."

VERO-16 Request-Level ClientTimeout Usage Extends Effective Timeouts	
Assets	src/providers/beacon_node.py src/providers/multi_beacon_node.py src/services/block_proposal.py src/providers/remote_signer.py
Status	Closed: See Resolution
Rating	Informational

Description

The request-level `ClientTimeout` passed by `BeaconNode.produce_block_v3()` specifies only the `connect` field, which causes `aiohttp` to silently replace the session-level `total=10` bound with its library default of 300 seconds.

In `aiohttp`, supplying a request-level `ClientTimeout` object replaces the session timeout entirely rather than merging with it.

The first manifestation occurs in block production. The request-level `ClientTimeout` passed by `BeaconNode.produce_block_v3()` specifies only the `connect` field. Because `ClientTimeout(connect=...)` is constructed without explicit `total` or `sock_read` values, the effective total timeout for block production requests rises from the intended 10 seconds to 5 minutes.

```
src/providers/beacon_node.py::BeaconNode.__init__()
# Session has bounded defaults
self.client_session = aiohttp.ClientSession(
    timeout=ClientTimeout(connect=1, total=10),
)

# ... snip ...

# produce_block_v3() overrides with connect-only timeout
resp = await self._make_request(
    method="GET",
    endpoint="/eth/v3/validator/blocks/{slot}",
    formatted_endpoint_string_params=dict(slot=slot),
    params=params,
    timeout=ClientTimeout(
        connect=self.client_session.timeout.connect, # @audit total/sock_read unset; session total=10 is replaced
    ),
)
```

The second manifestation occurs in the signing paths. The `BlockProposalService` functions `_fetch_randao_reveal()`, `_get_randao_reveal()`, and `_sign_block()` call `RemoteSigner.sign()` without explicit per-call deadlines. Since the high priority signer session also relies on `aiohttp` defaults, these requests can likewise wait up to 300 seconds.

```
src/services/block_proposal.py + src/providers/remote_signer.py
```

```
# Proposal-path calls do not pass per-call timeouts
_, randao_reveal, _ = await self.signature_provider.sign(...)
_, signature, _ = await self.signature_provider.sign(...)

# High-priority signer session has no explicit timeout configured
self.high_priority_client_session = aiohttp.ClientSession(
    base_url=self.url,
    headers=headers,
    trace_configs=[...],
)

# sign() also posts without a per-request timeout override
async with self._get_session_for_message(message).post(
    _endpoint.format(identifier=identifier),
    headers={ACCEPT: ContentType.TEXT_PLAIN.value},
    data=self.json_encoder.encode(message),
) as resp:
    ...
```

For the `produce_block_v3()` manifestation, this has negligible operational impact in practice. The fallback path in `MultiBeaconNode._produce_best_block()` that awaits these requests only executes when no beacon node returned a valid block within the initial timed window. At that point the proposer has already missed the slot regardless of whether the pending requests are cancelled promptly or allowed to linger. Additionally, each slot's proposal duty runs as an independent `asyncio` task created via `task_manager.create_task()`, so a stalled task for one slot cannot block proposals for subsequent slots. Remaining pending tasks are also explicitly cancelled once the fallback loop exits.

For the signing manifestation, this is primarily a timeout policy inconsistency that can affect liveness rather than a direct safety flaw.

This issue is rated as informational because both manifestations represents an unintended timeout configuration rather than a meaningful liveness or safety risk. The mismatch between the intended session timeout and the effective request timeout is a code quality concern worth correcting.

Recommendations

For `produce_block_v3()`, consider passing explicit `total` and `sock_read` values in the request-level `ClientTimeout` within `produce_block_v3()` so that the session's bounded defaults are preserved.

For signing paths, add explicit timeout/deadline handling for `RemoteSigner.sign()` calls in `BlockProposalService` steps, or set explicit bounded timeouts on the high-priority signer client session.

Resolution

The development team has acknowledged the issue with the following comment:

"The `total` / `sock_read` timeout values are intentionally omitted in block production API requests to allow beacon nodes ample time to respond even in suboptimal network conditions (e.g. non-finalization) when beacon nodes may take significantly longer to respond than usual. The session's bounded default timeout of 10 seconds is intentionally not used since it could be insufficient during those harder network conditions. The fallback default of 5 minutes is indeed excessive and could probably be lowered to one minute but this does not meaningfully change things.

For signing requests, a similar principle applies where we prefer to wait a little longer for a signature and submitting a duty late vs cancelling it prematurely. We agree that the fallback 5 minute default is slightly excessive. This

timeout was adjusted to 5 and 10 seconds for the high- and low-priority client sessions respectively as part of addressing the [VERO-05](#) finding."

VERO-17	Miscellaneous General Comments
Assets	/*
Status	Resolved: See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Inefficient `best_beacon_node` Property Implementation

Related Asset(s): `src/providers/multi_beacon_node.py`

The `best_beacon_node` property uses an inefficient algorithm that sorts the entire list of beacon nodes when only the maximum-scored node is needed.

The current implementation is as follows:

```
src/providers/multi_beacon_node.py::MultiBeaconNode.best_beacon_node()

@property
def best_beacon_node(self) -> BeaconNode:
    return next(
        bn
        for bn in sorted(
            self.initialized_beacon_nodes, key=lambda bn: bn.score, reverse=True
        )
    )
```

This implementation sorts the entire list of initialised beacon nodes by score in descending order, then returns only the first element using `next()`. The sorting operation has $O(n \log n)$ time complexity, where n is the number of beacon nodes. Since only the beacon node with the highest score is required, sorting the remaining elements is unnecessary work.

This issue is rated as informational as it represents a code quality concern with minor performance implications. The function executes quickly even with the inefficient implementation due to the typically small number of beacon nodes in production deployments.

Replace the sorting approach with Python's built-in `max()` function to find the beacon node with the highest score. This reduces the time complexity from $O(n \log n)$ to $O(n)$.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team's responses to the raised issues above are as follows:

1. [9504eaa](#): Replaced sorting approach with `max()` as recommended.

All miscellaneous comments have been addressed.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact			
	Likelihood		
	Low	Medium	High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'